Software Engineering - Lab 7

Fuzzing for Input Validation Bugs

Aditya Umashankar
PES1UG24AM335
12 November 2026

Assigned application: Coupon Validator

Objective: complete property-based fuzz tests for the three assigned functions, reproduce validation failures, harden the application, and verify the fixes.

Source and filename mapping

Source repository: SETAPESU26/L6_L7_31_coupon_validator, lab7 folder.
Source revision: d2005e7d222c58794044b29565b7d1f4439d5498.

The handout calls the application processor.py and the tests test_processor.py. The assigned repository uses coupon_validator.py and test_coupon_validator.py. Both command forms are provided; the compatibility files use the same implementation and tests.

Execution environment

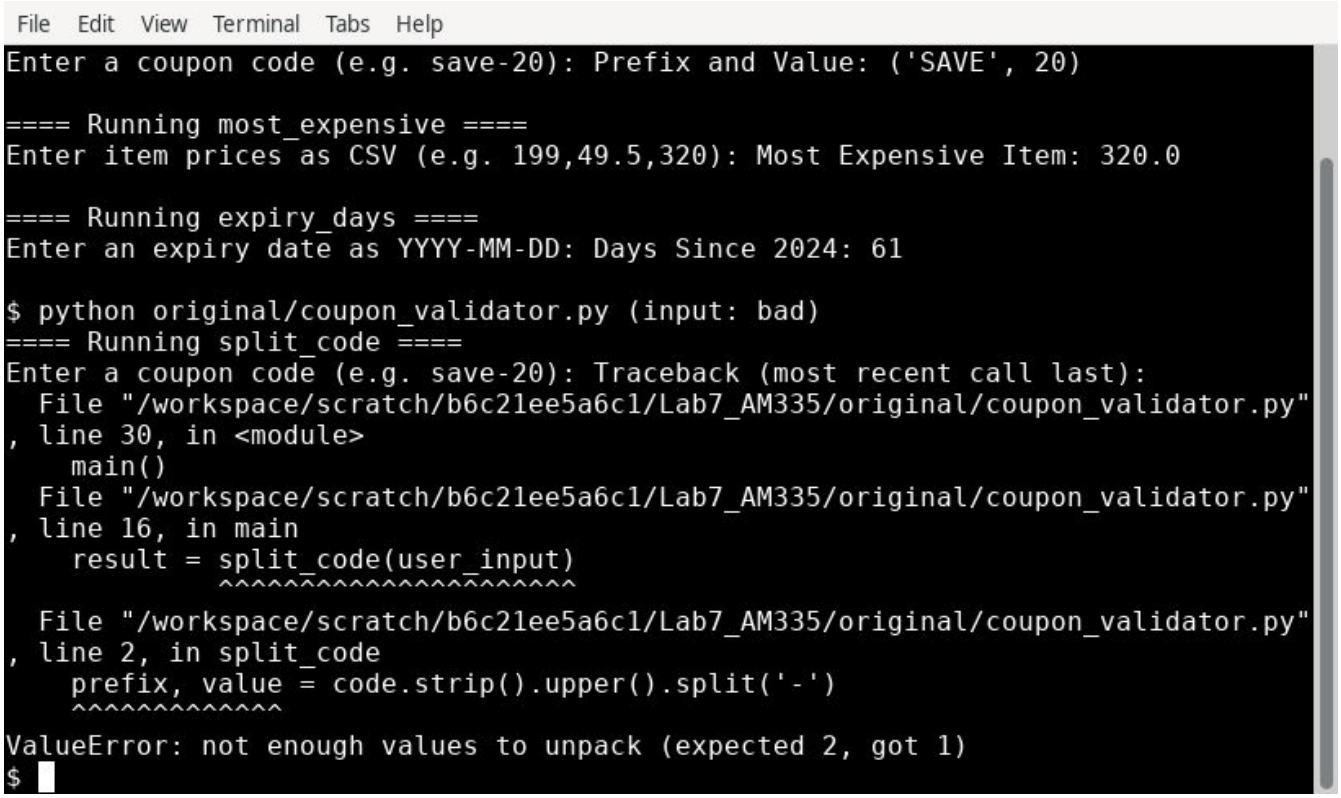
Python 3.12; pytest 9.2.0; Hypothesis 6.170.15. Screenshots are actual Linux terminal captures. External pytest plugins were disabled for the evidence run. The unchanged source files and execution logs are included.

Result

Original application: three no-crash properties failed.
Fixed application: **70 tests passed**. Six generated properties use up to 300 examples each; pytest test counts do not represent the number of generated inputs.

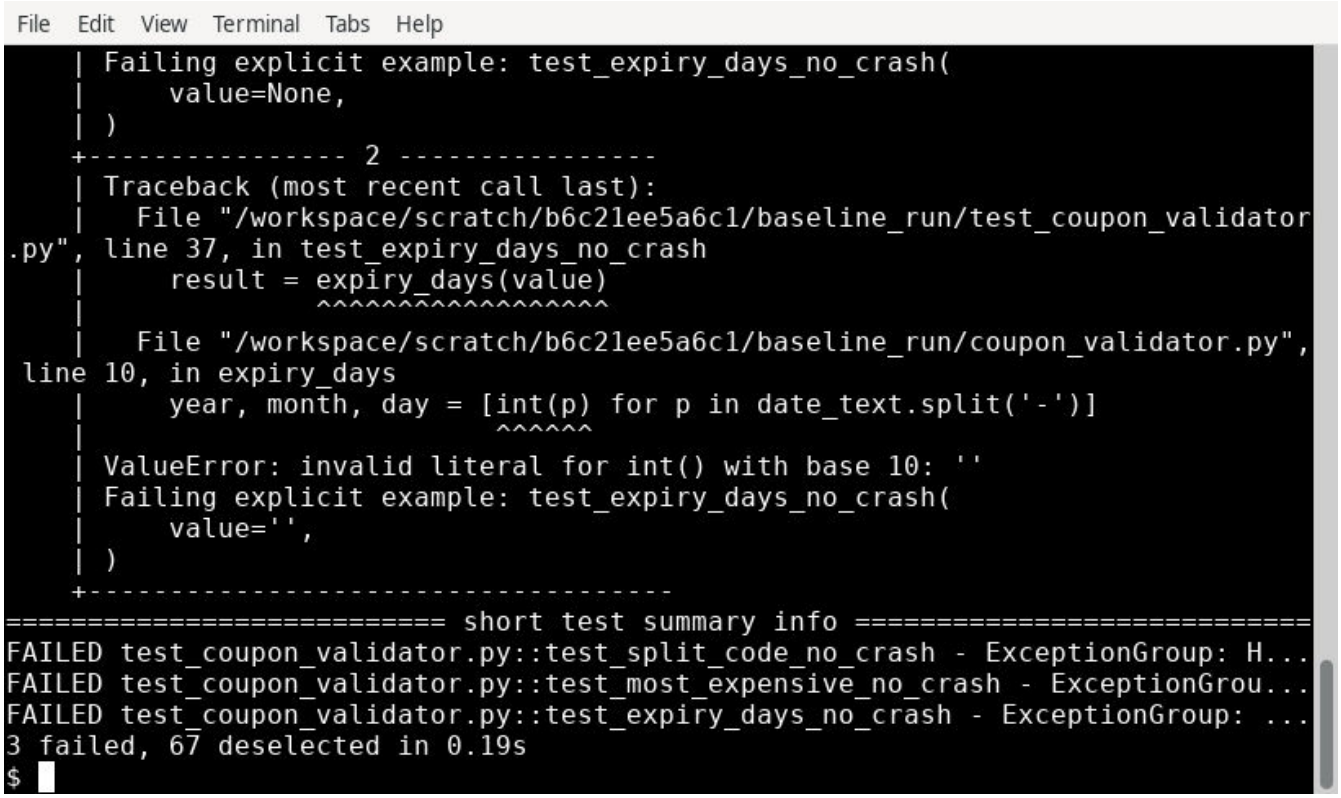
Step 2 - Run the original application

The valid coupon and price list work. The original returns 61 for 2024-03-01. Entering bad as the coupon raises ValueError and terminates execution. Inputs were supplied through standard input to make the runs reproducible.



Steps 3-4 - Complete and run fuzz tests

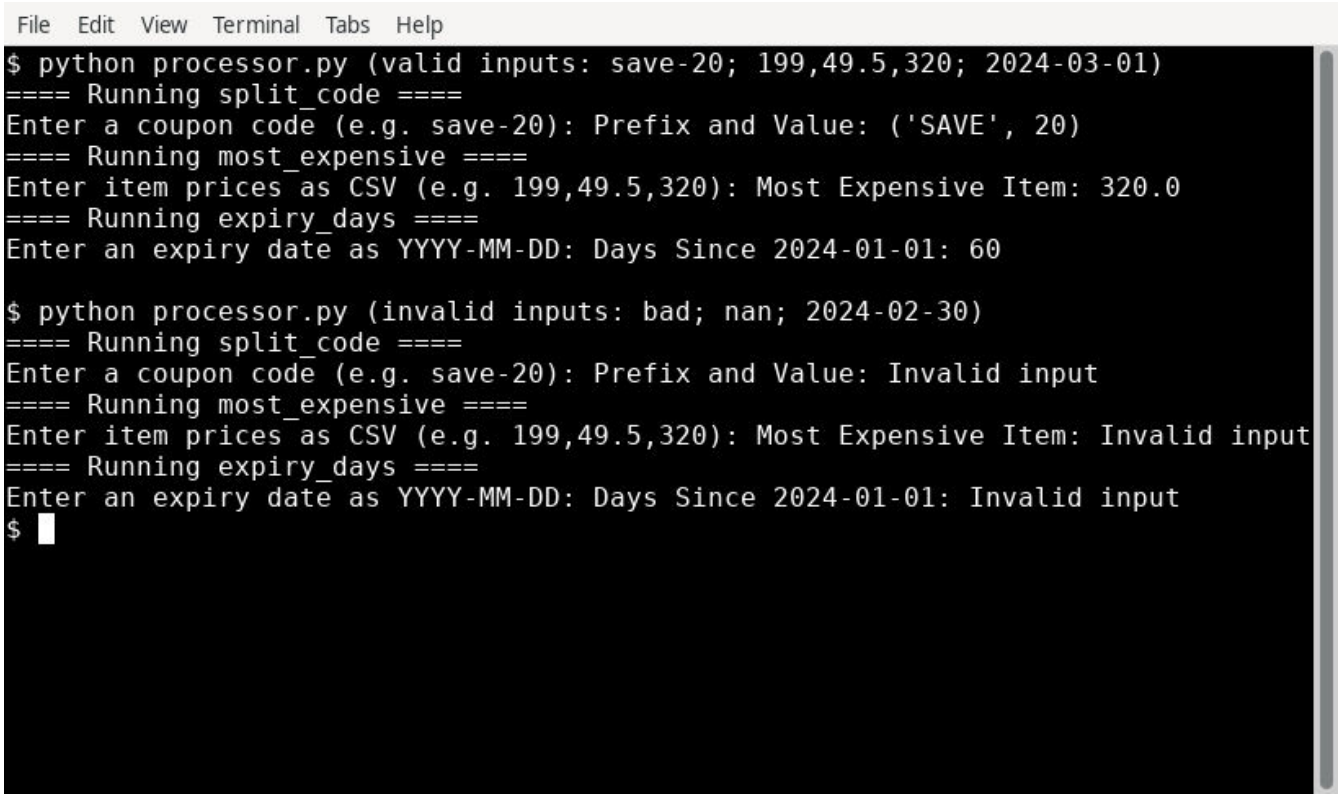
The completed no-crash properties fail on all three original functions: 3 failed, 67 deselected. None and the empty string trigger AttributeError or ValueError. Explicit examples retain these regressions alongside generated inputs. The same three tests are retained in the final suite.



The full failure trace is preserved in evidence/logs/02_failing_tests.txt. This screenshot shows the end of the trace and the failing summary.

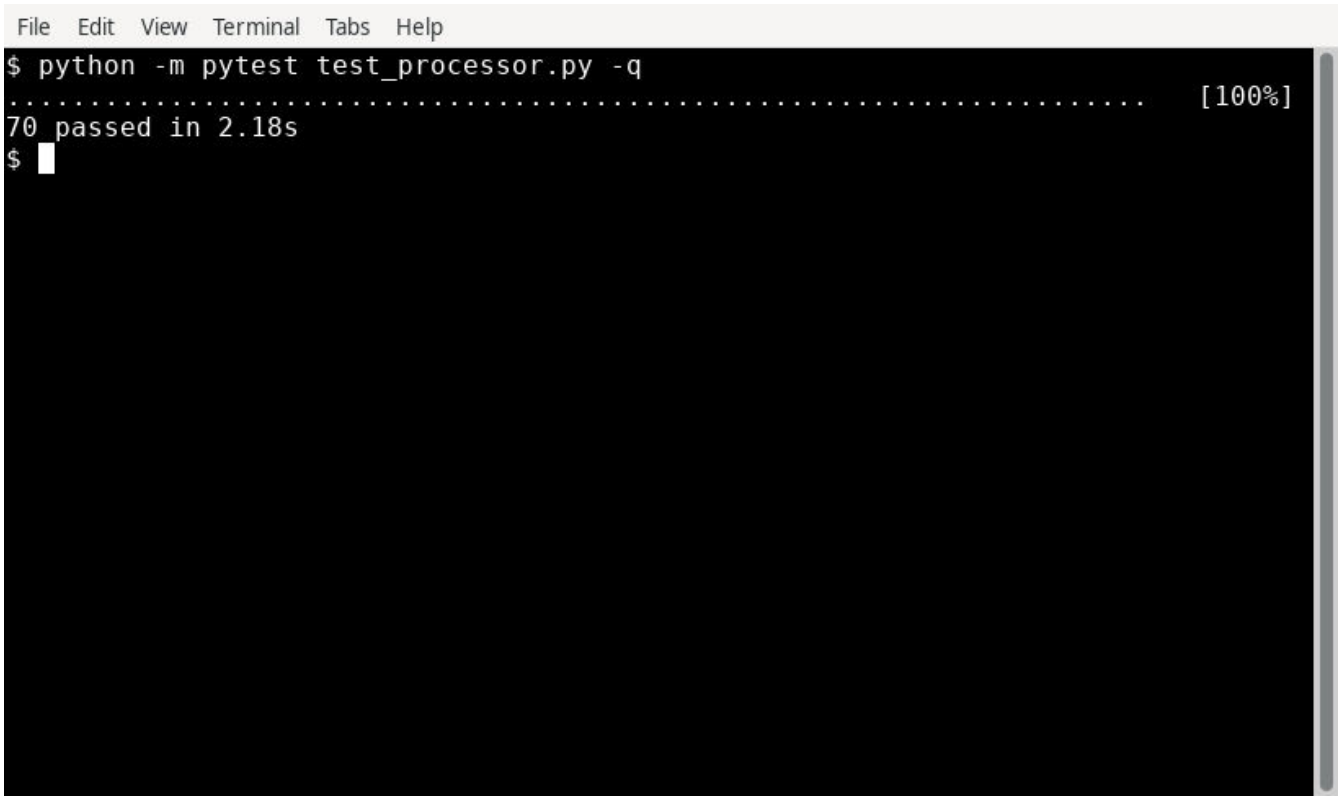
Step 5 - Run the fixed application

Valid inputs still produce (SAVE, 20) and 320.0. The date now returns 60 elapsed days. Malformed coupon, NaN price and impossible date inputs produce Invalid input; the CLI continues through all three functions.



Step 6 - Re-run the completed tests

The handout command python -m pytest test_processor.py -q completes successfully: 70 passed. The canonical test_coupon_validator.py command also passed. No tests are marked xfail or skipped to obtain this result.



Properties: arbitrary-input robustness for each function, valid coupon round trips, price maxima independent of CSV order, and calendar differences against datetime.date. Regression tests add malformed fields, Unicode, oversized input, nonfinite prices, leap days, boundary dates and CLI error handling.

Fixes and validation decisions

Coupon parsing

Require ASCII letters, one hyphen and 1-100 ASCII digits. Normalize the prefix to uppercase. Reject invalid types, missing fields, extra separators, negative amounts and oversized values.

Cart prices

Require a nonempty CSV with no empty tokens. Accept finite nonnegative decimal numbers, with an optional plus sign or exponent. Reject NaN, infinities, overflow, negatives and nonnumeric text. Preserve the original float result type.

Calendar dates

Require a valid zero-padded YYYY-MM-DD date. Calculate (date - 2024-01-01).days using datetime.date. The baseline is day zero, leap years are handled correctly and dates before the baseline return negative differences.

Failure behavior

Return None for invalid input. The CLI displays Invalid input and continues. EOF and keyboard cancellation end the session cleanly. String input is limited to 4096 characters as a defensive design choice.

Calendar checks

2024-01-01 = 0; 2024-02-29 = 59; 2024-03-01 = 60; 2025-01-01 = 366; 2023-12-31 = -1. Reject 2023-02-29 and 2024-04-31.

Reproduction

Install requirements.txt, then run python reproduce.py from the submission folder. It uses an isolated temporary copy of the original, checks the expected failing exit code, and reruns the fixed suite. See README.md for normal execution commands.

Scope

Returning None, ASCII-only coupon syntax, nonnegative prices and input bounds are the validation policy selected for this lab; the starter did not supply a full business specification. The date function measures elapsed days from the fixed baseline, not days remaining from today. Fuzzing improves coverage but does not prove the absence of all bugs.